In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

# ============================================================
# 2. Load the California Housing dataset
# ============================================================

print("Loading California Housing dataset...")

housing = fetch_california_housing(as_frame=True)
df = housing.frame

print("\nFirst five rows:")
print(df.head())

print("\nDataset shape:", df.shape)

# ============================================================
# 3. Select one feature and the target variable
# ============================================================

# AveRooms: Average number of rooms per dwelling
# MedHouseVal: Median house value, measured in units of $100,000

X = df[["AveRooms"]].values
y = df["MedHouseVal"].values

# Remove missing or infinite values, if any
valid = np.isfinite(X[:, 0]) & np.isfinite(y)
X = X[valid]
y = y[valid]

print("\nFeature: AveRooms")
print("Target: MedHouseVal")
print("Number of samples:", len(y))

# ============================================================
# 4. Split the data into training and testing sets
# ============================================================

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("\nTraining samples:", len(X_train))
print("Testing samples :", len(X_test))

# ============================================================
# 5. Standardize the feature using training data only
# ============================================================
```

```
# Standardization improves numerical stability for gradient descent
mean_X = X_train.mean()
std_X = X_train.std()

if std_X == 0:
    raise ValueError("The selected feature has zero variance.")

X_train_scaled = (X_train - mean_X) / std_X
X_test_scaled = (X_test - mean_X) / std_X

# Add a column of ones for the intercept
X_train_b = np.c_[np.ones(len(X_train_scaled)), X_train_scaled]
X_test_b = np.c_[np.ones(len(X_test_scaled)), X_test_scaled]

# ============================================================
# 6. Implement Linear Regression using Gradient Descent
# ============================================================

def gradient_descent(X, y, learning_rate=0.01, iterations=3000):
    """
    Minimize mean squared error using batch gradient descent.
    X must include a column of ones for the intercept.
    """

    m, n = X.shape
    theta = np.zeros(n)

    for i in range(iterations):

        # Predictions
        predictions = X @ theta

        # Calculate errors
        errors = predictions - y

        # Calculate gradients
        gradients = (2 / m) * (X.T @ errors)

        # Update parameters
        theta = theta - learning_rate * gradients

    return theta


print("\nTraining using Gradient Descent...")

theta_gd = gradient_descent(
    X_train_b,
    y_train,
    learning_rate=0.01,
    iterations=3000
)

# Predict using Gradient Descent
y_pred_gd = X_test_b @ theta_gd

# ============================================================
# 7. Implement Linear Regression using Normal Equation
# ============================================================
```

```
print("Training using the Normal Equation...")

# theta = (X transpose X)^(-1) X transpose y
# np.linalg.pinv is more numerically robust than directly inverting X.T @

theta_ne = np.linalg.pinv(X_train_b) @ y_train

# Predict using Normal Equation
y_pred_ne = X_test_b @ theta_ne

# ============================================================
# 8. Evaluate both models
# ============================================================

mse_gd = mean_squared_error(y_test, y_pred_gd)
r2_gd = r2_score(y_test, y_pred_gd)

mse_ne = mean_squared_error(y_test, y_pred_ne)
r2_ne = r2_score(y_test, y_pred_ne)

results = pd.DataFrame({
    "Method": ["Gradient Descent", "Normal Equation"],
    "MSE": [mse_gd, mse_ne],
    "R-squared": [r2_gd, r2_ne]
})

print("\nMODEL PERFORMANCE COMPARISON")
print("=" * 50)
print(results.round(4).to_string(index=False))

# ============================================================
# 9. Convert coefficients to original feature units
# ============================================================

# Convert standardized coefficients back to the original AveRooms scale
slope_gd = theta_gd[1] / std_X
intercept_gd = theta_gd[0] - slope_gd * mean_X

slope_ne = theta_ne[1] / std_X
intercept_ne = theta_ne[0] - slope_ne * mean_X

print("\nGRADIENT DESCENT EQUATION")
print(
    f"MedHouseVal = {intercept_gd:.4f} "
    f"+ ({slope_gd:.4f} * AveRooms)"
)

print("\nNORMAL EQUATION")
print(
    f"MedHouseVal = {intercept_ne:.4f} "
    f"+ ({slope_ne:.4f} * AveRooms)"
)

# ============================================================
# 10. Visualize the data and fitted regression lines
# ============================================================

# Sort points by feature value for smooth regression lines
sort_idx = np.argsort(X_test[:, 0])
```

```
X_plot = X_test[sort_idx]
y_plot = y_test[sort_idx]

y_line_gd = intercept_gd + slope_gd * X_plot[:, 0]
y_line_ne = intercept_ne + slope_ne * X_plot[:, 0]

plt.figure(figsize=(10, 6))

plt.scatter(
    X_plot[:, 0],
    y_plot,
    alpha=0.3,
    label="Actual test data"
)

plt.plot(
    X_plot[:, 0],
    y_line_gd,
    linewidth=2,
    label="Gradient Descent"
)

plt.plot(
    X_plot[:, 0],
    y_line_ne,
    linestyle="--",
    linewidth=2,
    label="Normal Equation"
)

plt.xlabel("Average Number of Rooms (AveRooms)")
plt.ylabel("Median House Value (units of $100,000)")
plt.title("Simple Linear Regression: California Housing Dataset")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# ============================================================
# 11. Conclusion
# ============================================================

print("\nCONCLUSION")
print("=" * 50)

print("Both methods implement simple linear regression.")

if abs(mse_gd - mse_ne) < 1e-4:
    print("Both methods produce very similar test MSE values.")
else:
    print("The test MSE values differ between the two methods.")

print(
    "Lower MSE indicates smaller prediction errors, while "
    "higher R-squared indicates that the model explains more "
    "of the variation in the target."
)

print(
    "Because only AveRooms is used, the model cannot account "
```

```
    "for other factors affecting house prices."
)
```

```
Loading California Housing dataset...

First five rows:
   MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Latitude
\
0  8.3252      41.0  6.984127   1.023810       322.0  2.555556     37.88
1  8.3014      21.0  6.238137   0.971880      2401.0  2.109842     37.86
2  7.2574      52.0  8.288136   1.073446       496.0  2.802260     37.85
3  5.6431      52.0  5.817352   1.073059       558.0  2.547945     37.85
4  3.8462      52.0  6.281853   1.081081       565.0  2.181467     37.85

   Longitude  MedHouseVal
0    -122.23        4.526
1    -122.22        3.585
2    -122.24        3.521
3    -122.25        3.413
4    -122.25        3.422

Dataset shape: (20640, 9)

Feature: AveRooms
Target: MedHouseVal
Number of samples: 20640

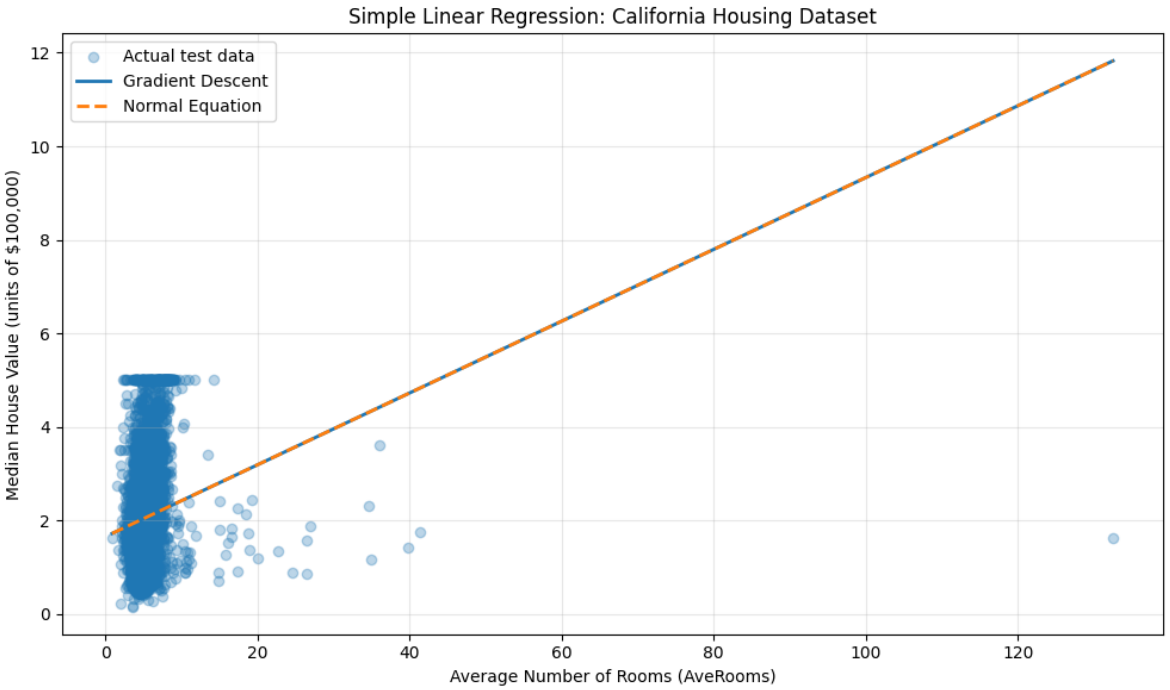
Training samples: 16512
Testing samples : 4128

Training using Gradient Descent...
Training using the Normal Equation...

MODEL PERFORMANCE COMPARISON
==================================================
          Method    MSE  R-squared
Gradient Descent 1.2923     0.0138
 Normal Equation 1.2923     0.0138

GRADIENT DESCENT EQUATION
MedHouseVal = 1.6548 + (0.0768 * AveRooms)

NORMAL EQUATION
MedHouseVal = 1.6548 + (0.0768 * AveRooms)
```




```
CONCLUSION
==================================================
Both methods implement simple linear regression.
Both methods produce very similar test MSE values.
Lower MSE indicates smaller prediction errors, while higher R-squared indi
cates that the model explains more of the variation in the target.
Because only AveRooms is used, the model cannot account for other factors
affecting house prices.
```

In [ ]: